Transform

The space transformation calculation mainly depend on the conception and theory of linear algebra and analytic geometry in space , like vector, matrix, cross product and dot product, etc. In computer graphics, the space transformation is a basic operation which can approximately be divided into two major groups, general and special. The general space transformation discuss translation, rotation and scaling. Thereinto, the rotation include the around fixed axis, the about an arbitrary axis, the Euler transform, the local coordination system alignment issue etc. The special space transformation refers to world transform, view space transform, clip space transform, tangent space transform, the transform from screen space to world space and so on. Those details is discussed below.

1. Mathematical Tool

1.1 Rectangular Coordinate System, Vector and Matrix

Three orthogonal unit vectors each other that pass through same point erect a rectangular coordinate system in 3D space. A component of a arbitrary vector is the length which projects on a certain axis of the coordinate system. Depending on it, the transformation matrix of coordinates can be constructed which change a vector from one coordinate system into another.

Assume three unit vectors $\mathbf{l}$, $\mathbf{u}$, $\mathbf{f}$ are orthogonal each other in the world space which create a LUF local space, $\mathbf{v}$ is arbitrary vector in the world space, as follow:

$$\mathbf{l} = \begin{bmatrix} l_x & l_y & l_z \end{bmatrix}^T, \quad \mathbf{u} = \begin{bmatrix} u_x & u_y & u_z \end{bmatrix}^T, \quad \mathbf{f} = \begin{bmatrix} f_x & f_y & f_z \end{bmatrix}^T$$

$$\mathbf{v} = [v_x \quad v_y \quad v_z]$$

Then, the vector transform formula can be built like this:

$$\mathbf{M}_v = \begin{bmatrix} \mathbf{l} & \mathbf{u} & \mathbf{f} \end{bmatrix} = \begin{bmatrix} l_x & u_x & f_x \\ l_y & u_y & f_y \\ l_z & u_z & f_z \end{bmatrix},$$

$$\mathbf{v}' = \mathbf{v} \cdot \mathbf{M}_v = \begin{bmatrix} v_x & v_y & v_z \end{bmatrix} \cdot \begin{bmatrix} l_x & u_x & f_x \\ l_y & u_y & f_y \\ l_z & u_z & f_z \end{bmatrix} = \begin{bmatrix} x & y & z \end{bmatrix} \cdot \begin{bmatrix} l_x & u_x & f_x \\ l_y & u_y & f_y \\ l_z & u_z & f_z \end{bmatrix}$$

$$= \begin{bmatrix} \mathbf{v} \cdot \mathbf{l} & \mathbf{v} \cdot \mathbf{u} & \mathbf{v} \cdot \mathbf{f} \end{bmatrix} = \begin{bmatrix} a & b & c \end{bmatrix}$$

Where $\mathbf{v}'$ is the LUF local space vector which is transformed from the vector $\mathbf{v}$ in the world space (refer to Figure 1). In Figure 1, If the vector $\mathbf{v}$,in the world space, project into LUF local space, it will correspond with the vector $\mathbf{v}'$.

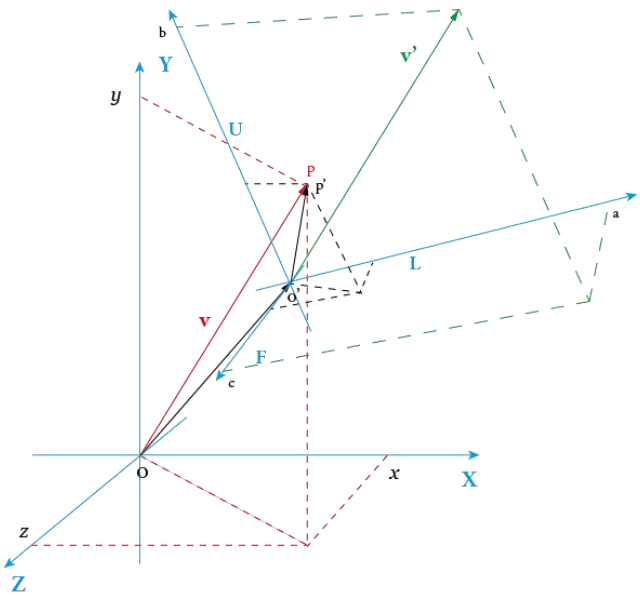

Figure 1.

Assume $\mathbf{p}$ is a point in the world space (refer to Figure 1), then the formula is:

$$\mathbf{l}=\begin{bmatrix} l_x & l_y & l_z & 0 \end{bmatrix}^T,\ \mathbf{u}=\begin{bmatrix} u_x & u_y & u_z & 0 \end{bmatrix}^T,\ \mathbf{f}=\begin{bmatrix} f_x & f_y & f_z & 0 \end{bmatrix}^T$$

$$\mathbf{p}=\begin{bmatrix} p_x & p_y & p_z & 1 \end{bmatrix},\ \mathbf{i}=\begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}^T$$

$$\mathbf{M}_{v4}=\begin{bmatrix} \mathbf{l} & \mathbf{u} & \mathbf{f} & \mathbf{i} \end{bmatrix}=\begin{bmatrix} l_x & u_x & f_x & 0 \\ l_y & u_y & f_y & 0 \\ l_z & u_z & f_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\because \mathbf{p}'=(\mathbf{p}-\mathbf{o}')\cdot\mathbf{M}_{v4}=\begin{bmatrix} (\mathbf{p}-\mathbf{o}')\cdot\mathbf{l} & (\mathbf{p}-\mathbf{o}')\cdot\mathbf{u} & (\mathbf{p}-\mathbf{o}')\cdot\mathbf{f} & 1 \end{bmatrix}$$

$$\mathbf{p}'=\mathbf{p}\cdot\mathbf{M}_p$$

$$\therefore \mathbf{p}\cdot\mathbf{M}_p=\begin{bmatrix} \mathbf{p}\cdot\mathbf{l}-\mathbf{o}'\cdot\mathbf{l} & \mathbf{p}\cdot\mathbf{u}-\mathbf{o}'\cdot\mathbf{u} & \mathbf{p}\cdot\mathbf{f}-\mathbf{o}'\cdot\mathbf{f} & 1 \end{bmatrix}$$

$$\therefore \mathbf{M}_p=\begin{bmatrix} l_x & u_x & f_x & 0 \\ l_y & u_y & f_y & 0 \\ l_z & u_z & f_z & 0 \\ -\mathbf{o}'\cdot\mathbf{l} & -\mathbf{o}'\cdot\mathbf{u} & -\mathbf{o}'\cdot\mathbf{f} & 1 \end{bmatrix}$$

Where, $\mathbf{M}_{v4}$ is the homogeneous matrix of $\mathbf{M}_v$, $\mathbf{p}'$ is that point $\mathbf{p}$ present in LUF coordinate system, $\mathbf{M}_p$ is the transform matrix from world space into local space (LUF). If setup the 4th component of $\mathbf{p}$ to zero, then $\mathbf{M}_p$ can be used to transform direction vector in homogeneous space as well.

Another interesting issue is how a local space point is transformed into world space if learning LUF base vector coordinate in the world space. It means that compute the inverse matrix of $\mathbf{M}_p$

from known $\mathbf{l}$, $\mathbf{u}$, $\mathbf{f}$, $\mathbf{o}'$. Obviously, $\mathbf{M}_p$ is not orthogonal matrix, solution of inverse matrix need use the corresponding adjoint matrix, that is very complex. In fact, if the projection of three base vectors, **x**, **y**, **z** from world space into LUF space can be computed, the inverse matrix will be obtained by same solving of $\mathbf{M}_p$. Observing, the projection of two vector each other have same length and the plus-minus sign, the coordinate of **o**' in world space just is opposite to **o** in LUF space. Base on this two cases, $\mathbf{M}_p^{-1}$ can be obtained more easily.

$$\mathbf{x} = [l_x \quad u_x \quad f_x \quad 0]^T,\ \mathbf{y} = [l_y \quad u_y \quad f_y \quad 0]^T,\ \mathbf{z} = [l_z \quad u_z \quad f_z \quad 0]^T$$

$$\mathbf{o}'' = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix} \cdot \mathbf{M}_p = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} l_x & u_x & f_x & 0 \\ l_y & u_y & f_y & 0 \\ l_z & u_z & f_z & 0 \\ -\mathbf{o}'\cdot\mathbf{l} & -\mathbf{o}'\cdot\mathbf{u} & -\mathbf{o}'\cdot\mathbf{f} & 1 \end{bmatrix}$$

$$= \begin{bmatrix} -\mathbf{o}'\cdot\mathbf{l} & -\mathbf{o}'\cdot\mathbf{u} & -\mathbf{o}'\cdot\mathbf{f} & 1 \end{bmatrix}$$

$$\mathbf{M}'_p = \begin{bmatrix} l_x & l_y & l_z & 0 \\ u_x & u_y & u_z & 0 \\ f_x & f_y & f_z & 0 \\ -\mathbf{o}''\cdot\mathbf{x} & -\mathbf{o}''\cdot\mathbf{y} & -\mathbf{o}''\cdot\mathbf{z} & 1 \end{bmatrix}$$

$$Check\ Out: \quad \mathbf{M}_p \cdot \mathbf{M}'_p = \mathbf{M}'_p \cdot \mathbf{M}_p = \mathbf{I} \quad \Rightarrow \quad \mathbf{M}_p^{-1} = \mathbf{M}'_p$$

Where, $\mathbf{o}''$ is the LUF coordinate of original point $\mathbf{o}$, the deduction of $\mathbf{M}'_p$ is same $\mathbf{M}_p$.

1.2 Quaternion

For representing the rotation, quaternion has a more advantage than matrix.

Quaternion definition:

$$\dot{\mathbf{q}} = (q_v, q_w) = iq_x + iq_y + kq_z + q_w, \quad q_v = iq_x + jq_y + kq_z = (q_x,\ q_y,\ q_z) \quad (1.2\text{-}1)$$

$$i^2 = j^2 = k^2 = -1,\ ij = -ji = k,\ jk = -kj = i,\ ki = -ik = j$$

$$q_v : the\ imaginary\ part\ of\ \dot{\mathbf{q}} \qquad i, j, k :\ imaginary\ unit$$

$$q_w : the\ real\ part\ of\ \dot{\mathbf{q}}$$

Need attention, there are essential distinction between the imaginary units $i,\ j,\ k$ and vectors $\mathbf{i},\ \mathbf{j},\ \mathbf{k}$, although its are very similar in many ways e.g.

$$Let\ vectors\ \mathbf{i},\ \mathbf{j}\ and\ \mathbf{k}\ cross\ each\ other.$$

$$similar: \quad \mathbf{ij} = -\mathbf{ji} = \mathbf{k}, \mathbf{jk} = -\mathbf{kj} = \mathbf{i},\ \mathbf{ki} = -\mathbf{ik} = \mathbf{j}$$

$$ij = -ji = k,\ jk = -kj = i,\ ki = -ik = j$$

$$different: \ \mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = 0, \quad i^2 = j^2 = k^2 = -1$$

But because of a vector multiplying it's self becoming zero, the form of result of two quaternions timing can be simplified if the imaginary units are regarded as three unit vectors crossing each other, like below:

$$\dot{\mathbf{q}}\dot{\mathbf{r}} = (q_x i + q_y j + q_z k + q_w)(r_x i + r_y j + r_z k + r_w)$$
$$= (q_y r_z - q_z r_y)i + (q_z r_x - q_x r_z)j + (q_x r_y - q_y r_x)k + r_w(q_x i + q_y j + q_z k)$$
$$+ q_w(r_x i + r_y j + r_z k) + (q_w r_w - q_x r_x - q_y r_y - q_z r_z)$$
$$\textit{Let vectors } \mathbf{i}, \mathbf{j} \textit{ and } \mathbf{k} \textit{ cross each other},$$
$$\textit{vector } \mathbf{q} = q_x\mathbf{i} + q_y\mathbf{j} + q_z\mathbf{k}, \quad \textit{vector } \mathbf{r} = r_x\mathbf{i} + r_y\mathbf{j} + r_z\mathbf{k}$$
$$\textit{Let } q_v \Leftrightarrow \mathbf{q}, \ r_v \Leftrightarrow \mathbf{r}$$
$$\because \mathbf{q}\cdot\mathbf{r} = q_x r_x + q_y r_y + q_z r_z, \ \mathbf{q}\times\mathbf{r} = (q_y r_z - q_z r_y)\mathbf{i} + (q_z r_x - q_x r_z)\mathbf{j} + (q_x r_y - q_y r_x)\mathbf{k}$$
$$\therefore \dot{\mathbf{q}}\dot{\mathbf{r}} = \mathbf{q}\times\mathbf{r} + r_w\mathbf{q} + q_w\mathbf{r} + q_w r_w - \mathbf{q}\cdot\mathbf{r}$$

So, quaternion can be stated like this as well:

$$\dot{\mathbf{q}} = (q_v, q_w) = \mathbf{q} + q_w = (\mathbf{q}, q_w), \qquad (1.2\text{-}2)$$
$$\textit{where } \mathbf{q} \textit{ is vector and } \mathbf{q} = \mathbf{i}q_x + \mathbf{j}q_y + \mathbf{k}q_z,$$
$$(\mathbf{i}, \mathbf{j} \textit{ and } \mathbf{k} \textit{ are unit vector and cross each other})$$

With the form $(1.2\text{-}2)$, the operation of quaternion can be simplified according to the operation law of vector.

Other related definition:

$$\textit{Conjugate}: \quad \dot{\mathbf{q}}^* = (q_v, q_w)^* = (\mathbf{q}, q_w)^* = (-\mathbf{q}, q_w)$$
$$\textit{Normal}: \quad \|\dot{\mathbf{q}}\| = \sqrt{\dot{\mathbf{q}}^*\dot{\mathbf{q}}} = \sqrt{\dot{\mathbf{q}}\dot{\mathbf{q}}^*} = \sqrt{q_w^2 - \mathbf{q}\cdot(-\mathbf{q})} = \sqrt{q_x^2 + q_y^2 + q_z^2 + q_w^2}$$
$$\textit{Unit Quaternion}: \quad \|\dot{\mathbf{q}}\| = 1$$
$$\textit{or } \dot{\mathbf{q}} = \cos\theta + \mathbf{u}\sin\theta, \ \|\mathbf{u}\| = 1 \quad (\textit{i.e. } \mathbf{u} \textit{ is unit vector})$$
$$\textit{Inverse}: \quad \|\dot{\mathbf{q}}\|^2 = \mathbf{q}\mathbf{q}^* \ \Rightarrow \ \mathbf{q}^{-1} = \frac{\mathbf{q}^*}{\|\dot{\mathbf{q}}\|^2}$$

A special expression about quaternion $(1.2\text{-}3)$ can be used to simplified rotation transform:

$$Let\ \ \dot{\mathbf{q}} = (\mathbf{q}, q_w) \wedge \|\dot{\mathbf{q}}\| = 1,\ \ \dot{\mathbf{p}} = (\mathbf{p}, 0),\ then$$

$$\dot{\mathbf{q}}\dot{\mathbf{p}}\dot{\mathbf{q}}^{-1}$$

$$= (\mathbf{q} \times \mathbf{p} + q_w \mathbf{p} - \mathbf{q} \cdot \mathbf{p})(-\mathbf{q} + q_w) \qquad Note.\ this\ is\ two\ quaternion\ s\ \ multiplying.$$

$$= (q_w^2 - \mathbf{q} \cdot \mathbf{q})\mathbf{p} + 2(\mathbf{q} \cdot \mathbf{p})\mathbf{q} + 2q_w \mathbf{q} \times \mathbf{p} \qquad (1.2\text{-}3)$$

$$Let\ \mathbf{q} = t\mathbf{u} \wedge \|\mathbf{u}\| = 1,\ then$$

$$\dot{\mathbf{q}}\dot{\mathbf{p}}\dot{\mathbf{q}}^{-1} = (q_w^2 - (t\mathbf{u})^2)\mathbf{p} + 2(t\mathbf{u} \cdot \mathbf{p})t\mathbf{u} + 2q_w t\mathbf{u} \times \mathbf{p}$$

$$= (q_w^2 - t^2)\mathbf{p} + 2t^2(\mathbf{u} \cdot \mathbf{p})\mathbf{u} + 2q_w t\mathbf{u} \times \mathbf{p} \qquad (1.2\text{-}4)$$

$$Refer\ to\ expression\ \ 2.3.2\text{-}1\ (\mathbf{p}' = \mathbf{p}\cos\alpha + (\mathbf{p} \cdot \mathbf{l})\mathbf{l}(1 - \cos\alpha) + (\mathbf{p} \times \mathbf{l})\sin\alpha\ )$$

$$Let\ \ \mathbf{l} = \mathbf{u}\ \ then\ \ have\ \ a\ \ deduction\ \ below:$$

$$q_w^2 - t^2 = \cos\alpha,\ 2t^2 = 1 - \cos\alpha,\ 2q_w t = \sin\alpha\ \ (from\ \ simplifying\ \ 2q_w t = -\sin\alpha\ )$$

$$Solution.\ \ \ t = \sqrt{\frac{1 - \cos\alpha}{2}} = \sin\frac{\alpha}{2},\ \ q_w = \sqrt{\cos\alpha + t^2} = \sqrt{\frac{1 + \cos\alpha}{2}} = \cos\frac{\alpha}{2}$$

$$Obtain.\ \ \dot{\mathbf{q}} = \mathbf{q} + q_w = \mathbf{u}\sin\frac{\alpha}{2} + \cos\frac{\alpha}{2} \qquad (1.2\text{-}5)$$

$$Note.\ the\ orentation\ of\ \ \mathbf{u}\ is\ identical\ with\ \mathbf{q}.$$

The $(1.2\text{-}3)$ and $(1.2\text{-}5)$ mean the rotation of vector $\mathbf{p}$ about vector $\mathbf{q}$ can be presented by three quaternions multiplication.

2. Common Space Transformation

2.1 Coordination System Alignment

In Figure 1, if LUF coordination system is turned to $\mathbf{l}$ aligning $\mathbf{x}$, $\mathbf{u}$ overlapping $\mathbf{y}$, $\mathbf{f}$ overlaying $\mathbf{z}$, the below formula have to be obeyed.

$$\mathbf{l} \cdot \mathbf{M}_{ag} = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix},\ \ \mathbf{u} \cdot \mathbf{M}_{ag} = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix},\ \ \mathbf{f} \cdot \mathbf{M}_{ag} = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

$$\because \mathbf{l}, \mathbf{u}, \mathbf{f}\ \ cross\ \ each\ other\ \ and\ \ are\ \ unit\ \ vector.$$

$$\therefore \mathbf{M}_{ag} = \begin{bmatrix} l_x & u_x & f_x \\ l_y & u_y & f_y \\ l_z & u_z & f_z \end{bmatrix},\ \ \mathbf{M}_{ag}\ is\ orthogonal\ matrix.$$

$$\therefore \mathbf{M}_{ag}^{-1} = \mathbf{M}^T = \begin{bmatrix} l_x & l_y & l_z \\ u_x & u_y & u_z \\ u_x & u_y & u_z \end{bmatrix}$$

2.2 Translation

Translation is about space point primarily. In 4-Dimension homogeneous space, a point vector plus a off vector will generate a new location vector. Base on this, a translation computation is like:

$$\mathbf{p} = \begin{bmatrix} x & y & z & 1 \end{bmatrix}, \quad \mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ dx & dy & dz & 1 \end{bmatrix},$$

$$\mathbf{p}' = \mathbf{p} \cdot \mathbf{M} = \begin{bmatrix} x+dx & y+dy & z+dz & 1 \end{bmatrix}$$

Where $\mathbf{M}$ is the translation matrix.

2.3 Rotation

2.3.1 Rotation around base axis

In fact, a rotation matrix around a certain base axis can deduct two others.

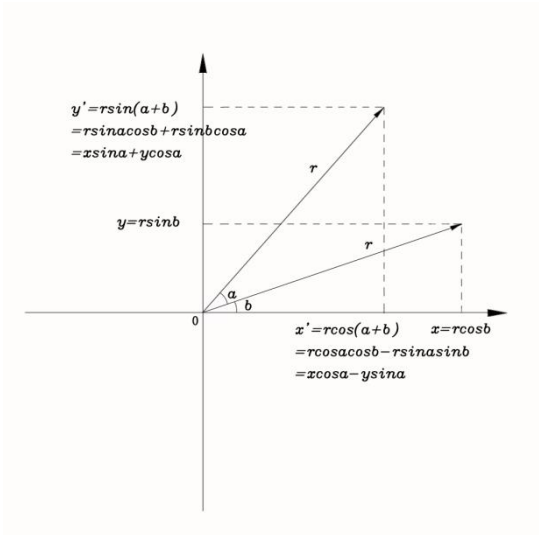

Figure 2.

Figure2 present a rotation about base axis $\mathbf{Z}$ which transform equation be written:

$$x' = x\cos a - y\sin a, \quad y' = x\sin a + y\cos a \quad (*)$$

$$\mathbf{p}' = \begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \mathbf{p} \cdot \mathbf{M}_{Rx} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot \mathbf{M}_{Rx}$$

$$\mathbf{M}_{Rz} = \begin{bmatrix} \cos a & \sin a & 0 & 0 \\ -\sin a & \cos a & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Move observer position to negative half $\mathbf{X}$ axis, the view from negative to positive. If the axis vector $\mathbf{Y}$ and $\mathbf{Z}$ rotate 90 degree clockwise, a new coordinate system is erected which axis vectors $\mathbf{X}$, $\mathbf{Y}$, $\mathbf{Z}$ match $\mathbf{Z}$, $\mathbf{X}$, $\mathbf{Y}$ in Figure 2 separately. It means the rotation matrix around $\mathbf{X}$ axis can be created from $\mathbf{M}_{Rx}$ like this:

$$y' = y\cos a - z\sin a, \quad z' = y\sin a + z\cos a$$

$$\mathbf{M}_{Rx} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos a & \sin a & 0 \\ 0 & -\sin a & \cos a & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Similarly, if observing from $\mathbf{Y}$ axis positive to negative and having other axes, $\mathbf{Z}$ and $\mathbf{X}$, matching $\mathbf{X}$ and $\mathbf{Y}$ in Figure 2 individually, the rotation matrix around $\mathbf{Y}$ can be created on this new coordinate system as below:

$$z' = z\cos a - x\sin a, \quad x' = z\sin a + x\cos a$$

$$\mathbf{M}_{Ry} = \begin{bmatrix} \cos a & 0 & -\sin a & 0 \\ 0 & 1 & 0 & 0 \\ \sin a & 0 & \cos a & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.3.2 Rotation around arbitrary axis

There are two methods for solving the rotation matrix around arbitrary axis.

Method A.

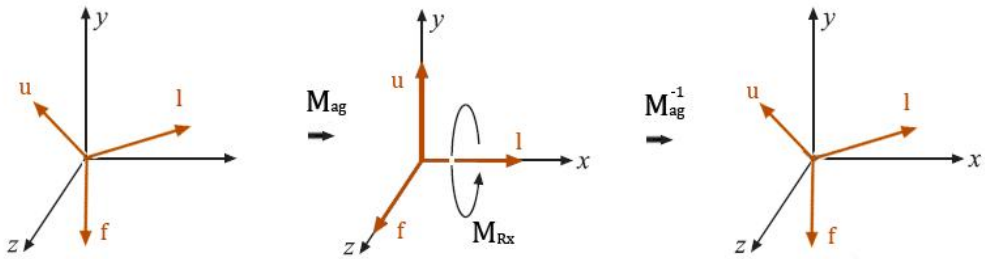

Figure 3.

Refer to Figure 3, suppose to solve a rotation around vector $\mathbf{l}$ which point arbitrary orientation. The procedure can be divided into three phases. First, erect a local rectangular coordinate system of which vector $\mathbf{l}$ is a axis, then align it and world coordinate system (employ the method in 2.1 to obtain the alignment matrix $\mathbf{M}_{ag}$). Second, operate with rotation around base axis (refer to 2.3.1) . Third, transform opposite aligning operation by matrix $\mathbf{M}_{ag}^{-1}$. The whole compute is:

$$\mathbf{M}_{Ra} = \mathbf{M}_{ag} \cdot \mathbf{M}_{R} \cdot \mathbf{M}_{ag}^{-1}$$

The local rectangular coordinate system which make the arbitrary vector into a base axis will be created by the following method:

$$\mathbf{u}=\begin{cases}[0,-l_z,l_y], & |l_x|<|l_y| \; and \; |l_x|<|l_z| \\ [-l_z,0,l_x], & |l_y|<|l_x| \; and \; |l_y|<|l_z| \\ [-l_y,l_x,0], & |l_z|<|l_x| \; and \; |l_z|<|l_y|\end{cases}$$

$$\mathbf{u}=\frac{\mathbf{u}}{\|\mathbf{u}\|}$$

$$\mathbf{f}=\mathbf{l}\times\mathbf{u}$$

Method B.

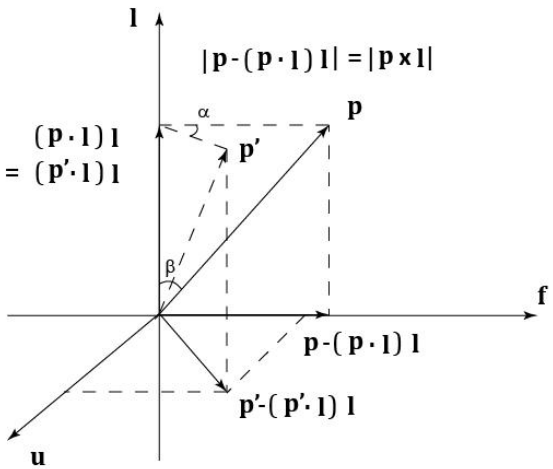

Figure 4.

Let $\mathbf{l}$ is the surrounded unit vector, unit vector $\mathbf{p}$ rotates about it. $\mathbf{l}$ is a arbitrary direction vector on which a rectangle coordinate system is created that is built by three base axis, $\mathbf{l}$, $\mathbf{u}$ and $\mathbf{f}$. (The method for generating $\mathbf{u}$ and $\mathbf{f}$ refer to Method A. In fact, the two vectors can be ignored for computing the rotation matrix and just be considered as an assistance for thinking) Let the radian between $\mathbf{p}$ and $\mathbf{l}$ is β, $\mathbf{p}$ become $\mathbf{p}'$ after rotating radian α around $\mathbf{l}$. (Refer to Figure 4.)

Aware of the relationship $\|\mathbf{p}-(\mathbf{p}\cdot\mathbf{l})\,\mathbf{l}\| = \|\mathbf{p}\times\mathbf{l}\|, \quad (\mathbf{p}\cdot\mathbf{l})\,\mathbf{l}=(\mathbf{p}'\cdot\mathbf{l})\,\mathbf{l}$, a formula can be obtained as below:

$$\mathbf{p}'-(\mathbf{p}'\cdot\mathbf{l})\mathbf{l}=\mathbf{f}\|\mathbf{p}-(\mathbf{p}\cdot\mathbf{l})\mathbf{l}\|\cos\alpha+\mathbf{u}\|\mathbf{p}-(\mathbf{p}\cdot\mathbf{l})\mathbf{l}\|\sin\alpha$$
$$=(\mathbf{p}-(\mathbf{p}\cdot\mathbf{l})\mathbf{l})\cos\alpha+\mathbf{u}\|\mathbf{p}\times\mathbf{l}\|\sin\alpha$$
$$=(\mathbf{p}-(\mathbf{p}\cdot\mathbf{l})\mathbf{l})\cos\alpha+(\mathbf{p}\times\mathbf{l})\sin\alpha$$
$$\therefore\ \mathbf{p}'=\mathbf{p}'-(\mathbf{p}'\cdot\mathbf{l})\mathbf{l}+(\mathbf{p}'\cdot\mathbf{l})\mathbf{l}=(\mathbf{p}-(\mathbf{p}\cdot\mathbf{l})\mathbf{l})\cos\alpha+(\mathbf{p}\times\mathbf{l})\sin\alpha+(\mathbf{p}\cdot\mathbf{l})\mathbf{l}$$
$$=\mathbf{p}\cos\alpha+(\mathbf{p}\cdot\mathbf{l})\mathbf{l}(1-\cos\alpha)+(\mathbf{p}\times\mathbf{l})\sin\alpha \qquad (2.3.2\text{-}1)$$

The vector dot is commutative but not associative, i.e. $\mathbf{A}\cdot\mathbf{B}=\mathbf{B}\cdot\mathbf{A}, \quad (\mathbf{A}\cdot\mathbf{B})\mathbf{C}\neq\mathbf{A}(\mathbf{B}\cdot\mathbf{C})$, so the method $(\mathbf{p}\cdot\mathbf{l})\mathbf{l}=\mathbf{p}(\mathbf{l}\cdot\mathbf{l})$ is wrong if separating $\mathbf{p}$ from the second item of expression $(2.3.2-1)$. The correct path is like this:

$$(\mathbf{p}\cdot\mathbf{l})\mathbf{l} = (p_x l_x^2 + p_y l_x l_y + p_z l_x l_z,\ p_x l_x l_y + p_y l_y^2 + p_z l_x l_z,\ p_x l_x l_z + p_y l_y l_z + p_z l_z^2)$$

$$= \mathbf{p}\begin{bmatrix} l_x^2 & l_x l_y & l_x l_z \\ l_x l_y & l_y^2 & l_y l_z \\ l_x l_z & l_x l_z & l_z^2 \end{bmatrix}$$

So, the equation below can be obtained:

$$\mathbf{p}' = \mathbf{p}\cos\alpha\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} + \mathbf{p}\begin{bmatrix} l_x^2 & l_x l_y & l_x l_z \\ l_x l_y & l_y^2 & l_y l_z \\ l_x l_z & l_y l_z & l_z^2 \end{bmatrix}(1-\cos\alpha) + \mathbf{p}\begin{bmatrix} 0 & l_z & -l_y \\ -l_z & 0 & l_x \\ l_y & -l_x & 0 \end{bmatrix}\sin\alpha$$

$$(2.3.2\text{-}2)$$

2.3.3 Euler Transform

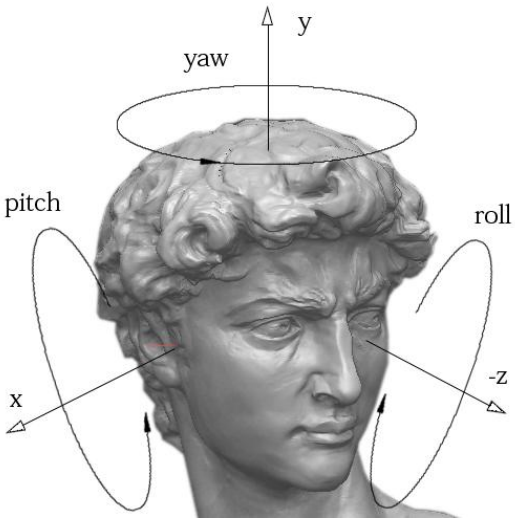

Figure 5

Euler transform, the method of stating and processing the rotation of point or rigid body in 3D space, is relatively more intuitive which is able to expediently adjust the object in 3D space to arbitrary posture . Here are a few things to look out for:

A. Traditionally, the Euler transform use a right-handed coordinate system of which the -z is the "positive" direction. The rotation around the x-axis, y-axis or z-axis also is called "pitch", "yaw", "roll" separately. (Refer to Figure 5)

B. An arbitrary rotation in 3D space can be regarded as a composition of three rotation around the respective Euler axis to cause the continued matrix multiplication, i.e. the Concatenation of Transform. Because of the commutative law can not be fulfilled in matrix multiplication, the final Euler matrix from concatenation of transform is related to the multiplication order which is variety of, even two Euler axis e.g. Z/Y/Z. The most common combination is Y/X/Z, i.e. first rotate around y-axis, then x-axis, final z-axis. The amount of the order combination is twelve.

C. Local Coordinate System & World Coordinate System
Euler transform is not on world coordinate system, but just on local coordinate system to change their axis orientation with rotation.

D. Gimbal Lock

Whatever order combination, Gimbal Lock is going to present as the second rotation angle equal $\frac{\pi}{2}+2k\pi, \quad (k \in Z)$. For example in Y/X/Z.

$$M_E = M_y M_x M_z = \begin{bmatrix} \cos y & 0 & -\sin y & 0 \\ 0 & 1 & 0 & 0 \\ \sin y & 0 & \cos y & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos x & \sin x & 0 \\ 0 & -\sin x & \cos x & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos z & \sin z & 0 & 0 \\ -\sin z & \cos z & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos y \cos z - \sin x \sin y \sin z & \cos y \sin z + \sin x \sin y \cos z & -\cos x \sin y & 0 \\ -\cos x \sin z & \cos x \cos z & \sin x & 0 \\ \sin y \cos z + \sin x \cos y \sin z & \sin y \sin z - \sin x \cos y \cos z & \cos x \cos y & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$Let \;\; x = \frac{\pi}{2} + 2k\pi \quad (k \in Z), \;\; then \;\; M_E = \begin{bmatrix} \cos(y+z) & \sin(y+z) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ \sin(y+z) & -\cos(y+z) & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$Let \;\; y = \frac{\pi}{2} + 2k\pi \quad (k \in Z), \;\; then \;\; M_E = \begin{bmatrix} -\sin x \sin z & \sin x \cos z & -\cos x & 0 \\ -\cos x \sin z & \cos x \cos z & \sin x & 0 \\ \cos z & \sin z & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$Let \;\; z = \frac{\pi}{2} + 2k\pi \quad (k \in Z), \;\; then \;\; M_E = \begin{bmatrix} -\sin x \sin y & \cos y & -\cos x \sin y & 0 \\ -\cos x & 0 & \sin x & 0 \\ \sin x \cos y & \sin y & \cos x \cos y & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

If the angle is $\frac{\pi}{2}$ in the second rotation about x-axis, the rotation matrix M_E just is controlled by two factors, the angle y around y-axis and angle z around z-axis. Whatever adjusting the third rotation parameter z it seems to be adjusting parameter y, i.e. it look like the z-axis is lost and the object is locked on y-axis. Analogous condition do not occurred as y or z equal $\frac{\pi}{2}$.

2.3.4 Quaternion & Rotation matrix

A. Get the rotation matrix from quaternion

From (1.2-4) $\dot{\mathbf{q}}\dot{\mathbf{p}}\dot{\mathbf{q}}^{-1} = (q_w^2 - t^2)\mathbf{p} + 2t^2(\mathbf{u}\cdot\mathbf{p})\mathbf{u} + 2q_w t\mathbf{u}\times\mathbf{p} \qquad (\mathbf{q} = t\mathbf{u} \ \wedge \ \|\mathbf{u}\| = 1)$

let $q_w^2 - t^2 = \cos\alpha,\ 2t^2 = 1-\cos\alpha,\ 2q_w t = \sin\alpha$

and (2.3.2-1), (2.3.2-2)

$$\mathbf{p}' = \mathbf{p}\cos\alpha + (\mathbf{p}\cdot\mathbf{l})\mathbf{l}(1-\cos\alpha) + (\mathbf{p}\times\mathbf{l})\sin\alpha$$

$$\mathbf{p}' = \mathbf{p}\cos\alpha\begin{bmatrix}1&0&0\\0&1&0\\0&0&1\end{bmatrix} + \mathbf{p}\begin{bmatrix}l_x^2 & l_xl_y & l_xl_z\\ l_xl_y & l_y^2 & l_yl_z\\ l_xl_z & l_yl_z & l_z^2\end{bmatrix}(1-\cos\alpha) + \mathbf{p}\begin{bmatrix}0 & l_z & -l_y\\ -l_z & 0 & l_x\\ l_y & -l_x & 0\end{bmatrix}\sin\alpha$$

As the vector $\mathbf{u}$ *considered as* $\mathbf{l}$, *a deduction is gotten*:

$$\dot{\mathbf{p}}' = \dot{\mathbf{q}}\dot{\mathbf{p}}\dot{\mathbf{q}}^{-1}$$

$$= \mathbf{p}\cos\alpha\begin{bmatrix}1&0&0\\0&1&0\\0&0&1\end{bmatrix} + \mathbf{p}\begin{bmatrix}u_x^2 & u_xu_y & u_xu_z\\ u_xu_y & u_y^2 & u_yu_z\\ u_xu_z & u_yu_z & u_z^2\end{bmatrix}(1-\cos\alpha) + \mathbf{p}\begin{bmatrix}0 & u_z & -u_y\\ -u_z & 0 & u_x\\ u_y & -u_x & 0\end{bmatrix}\sin\alpha$$

Replace with $\cos\alpha = q_w^2 - t^2, 1-\cos\alpha = 2t^2,\ \sin\alpha = 2q_w t$ *then*,

$$\dot{\mathbf{p}}' = \dot{\mathbf{q}}\dot{\mathbf{p}}\dot{\mathbf{q}}^{-1}$$

$$= \mathbf{p}\cos\alpha\begin{bmatrix}1&0&0\\0&1&0\\0&0&1\end{bmatrix} + \mathbf{p}\begin{bmatrix}u_x^2 & u_xu_y & u_xu_z\\ u_xu_y & u_y^2 & u_yu_z\\ u_xu_z & u_yu_z & u_z^2\end{bmatrix}(1-\cos\alpha) + \mathbf{p}\begin{bmatrix}0 & u_z & -u_y\\ -u_z & 0 & u_x\\ u_y & -u_x & 0\end{bmatrix}\sin\alpha$$

$$= \mathbf{p}\begin{bmatrix}q_w^2-t^2 & 0 & 0\\ 0 & q_w^2-t^2 & 0\\ 0 & 0 & q_w^2-t^2\end{bmatrix} + \mathbf{p}\begin{bmatrix}2t^2u_x^2 & 2t^2u_xu_y & 2t^2u_xu_z\\ 2t^2u_xu_y & 2t^2u_y^2 & 2t^2u_yu_z\\ 2t^2u_xu_z & 2t^2u_yu_z & 2t^2u_z^2\end{bmatrix}$$

$$+ \mathbf{p}\begin{bmatrix}0 & 2q_wtu_z & -2q_wtu_y\\ -2q_wtu_z & 0 & 2q_wtu_x\\ 2q_wtu_y & -2q_wtu_x & 0\end{bmatrix}$$

$$\because\ \dot{\mathbf{q}} = (\mathbf{q}, q_w) = (q_x, q_y, q_z, q_w) \ \Rightarrow\ q_x = tu_x,\ q_y = tu_y,\ q_z = tu_z$$

$$\|\dot{\mathbf{q}}\| = 1 \quad (i.e.\ q_x^2 + q_y^2 + q_z^2 + q_w^2 = 1) \Rightarrow q_w^2 - q_x^2 - q_y^2 - q_z^2 = 1 - 2q_x^2 - 2q_y^2 - 2q_z^2$$

$$\therefore\ \dot{\mathbf{p}}' = (\mathbf{p}', p_w) \ \wedge\ \mathbf{p}' = \mathbf{p}\begin{bmatrix}1-2q_y^2-2q_z^2 & -2q_zq_w+2q_xq_y & 2q_yq_w+2q_xq_z\\ 2q_zq_w+2q_xq_y & 1-2q_x^2-2q_z^2 & -2q_xq_w+2q_yq_z\\ -2q_yq_w+2q_xq_z & 2q_xq_w+2q_yq_z & 1-2q_x^2-2q_y^2\end{bmatrix}$$

B. Get the quaternion from rotation matrix

3. Special Space Transformation

Rendering Pipeline Transformation

3.1 The transformation from model space into world space

3.2 The transformation from world space into view space

3.2.1 Orthographic Projection

3.2.2 Perspective Projection

3.3 The transformation from view space into clip space (CVV)

3.4 The transformation from screen space into world space

Assume the original point of a screen space is left-top, screen width is w, screen high is h. Then the coordinate of four corners are (0, 0), (w, 0), (w, h), (0, h) in clockwise order (in Figure 3.4.1).

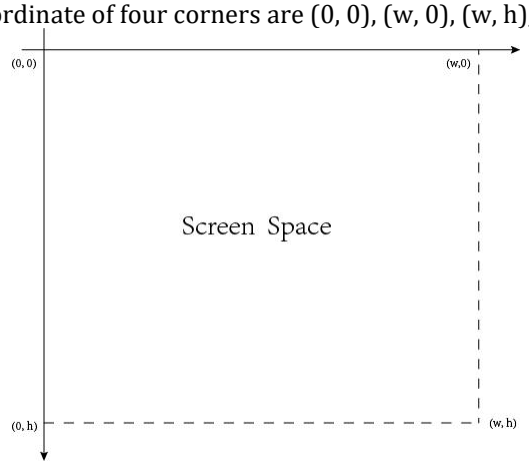

Figure 3.4.1

If this screen space will be projected into a rectangular coordinate system $(\mathbf{l}, \mathbf{u}, \mathbf{f})$ in world space and horizon orientation mapping the vector $\mathbf{l}$, vertical orientation mapping the vector $\mathbf{f}$ like Figure 3.4.2.

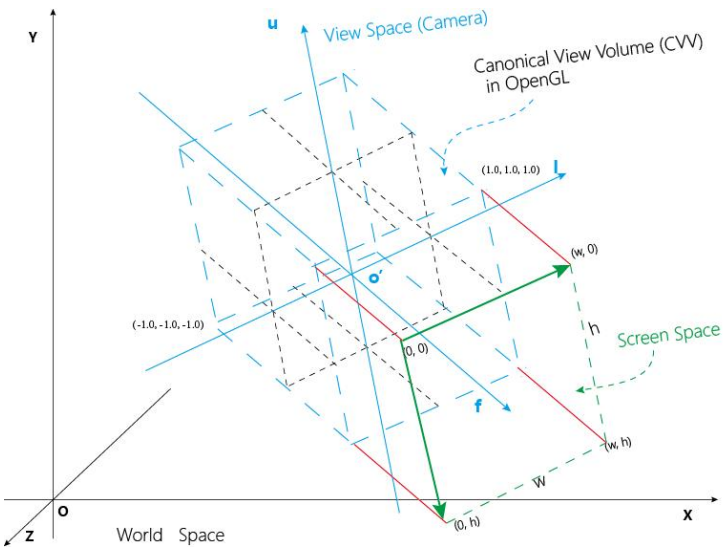

Figure 3.4.2

Commonly, coordinate system $(\mathbf{l},\mathbf{u},\mathbf{f})$ is View Space base (i.e. Camera Space coordinate system) coincidentally. CVV (Canonical View Volume) is in Camera Space as well. If Screen Space project into the $\mathbf{l}-\mathbf{u}$ plane of CVV in OpenGL, the transform matrix can be:

$$M_{S2V}=\begin{bmatrix} a & b & 0 & 0 \\ d & e & 0 & 0 \\ 0 & 0 & 0 & 0 \\ g & h & 0 & 1 \end{bmatrix} \quad \text{and} \quad \begin{aligned} &[0,0,t,1]\cdot M_{S2V}=[-1,\ 1,\ 0,\ 1] \\ &[W,0,t,1]\cdot M_{S2V}=[\ 1,\ 1,\ 0,\ 1] \\ &[0,H,t,1]\cdot M_{S2V}=[-1,-1,0,1] \\ &[W,H,t,1]\cdot M_{S2V}=[\ 1,-1,\ 0,1] \end{aligned}$$

t is optionally, set $t=0$, obtain :

$g=-1,\ h=1,$

$Wa+g=1,\ Wb+h=1,$

$Hd+g=-1,\ He+h=-1,$

$$\Rightarrow a=\frac{2}{W},\ b=0,\ d=0,\ e=-\frac{2}{H}$$

$$\therefore M_{S2V}=\begin{bmatrix} \frac{2}{W} & 0 & 0 & 0 \\ 0 & \frac{-2}{H} & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 1 & 0 & 1 \end{bmatrix}$$

Assume the transform matrix from Camera Space into CVV is M_{C2V}, from World Space into Camera Space is M_{W2C}, then obtain the transform matrix from Screen Space into World Space:

$$M_{S2W}=M_{S2V}\cdot M_{C2V}^{-1}\cdot M_{W2C}^{-1}$$

Actually, M_{W2C} is the view transform matrix and M_{C2V} is the projection matrix.

In Figure 3.4.2, the screen space original point is left-top corner . It's used in DirectX usually. In OpenGL, the location (0,0) is left-bottom corner. Assume render procedure is by OpenGL low and output is DirectX standard, the scene will be inverse up-down. It's similar for the transform from screen space into clip space(i.e. CVV). So, the screen coordinates need be changed into OpenGL style like:

$$M_{D2O} = \begin{bmatrix} a & b & 0 & 0 \\ c & d & 0 & 0 \\ 0 & 0 & 0 & 0 \\ e & f & 0 & 1 \end{bmatrix} \quad \text{and} \quad \begin{aligned} &[0,0,0,1]\cdot M_{D2O} = [0,H,0,1] \\ &[W,0,0,1]\cdot M_{D2O} = [W,H,0,1] \\ &[0,H,0,1]\cdot M_{D2O} = [0,0,0,1] \\ &[W,H,0,1]\cdot M_{D2O} = [W,0,0,1] \end{aligned}$$

Obtain:

$$e = 0, f = H,$$

$$a = 1,\ b = 0,$$

$$c = 0, d = -1,$$

$$M_{D2O} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & H & 0 & 1 \end{bmatrix}$$

$$\therefore M_{S2W} = M_{D2O} \cdot M_{S2V} \cdot M_{C2V}^{-1} \cdot M_{W2C}^{-1}$$

Another method is to get the matrix from CVV into Screen Space first, then compute the inverse matrix of it. The result is equally.